MIT 8103 ADVANCED DATABASE SYSTEMS

Individual Continuous Assessment Portfolio
Student: Precious Junior Frederick
Student ID: 301903679
Case Study: ArenaPulse Gaming
DBMS: PostgreSQL and MongoDB

This summary presents five practical portfolio activities for a Real-time Game Leaderboard. PostgreSQL stores the relational player, game, match, and score records; MongoDB demonstrates a separate event-document model. I personally ran and tested the database code and reviewed the outputs. Portfolio 5 is a distributed/cloud design discussion no cloud deployment or replica-set failover is claimed.

Portfolio 1: Database design and modelling

The PostgreSQL `portfolio` schema contains `player`, `game`, `match` and `score`. Primary and foreign keys connect the records. Unique constraints protect player handles, game titles and each `(match_id, player_id)` score pair; check constraints restrict match status and reject negative points. The design documentation, ER diagram, sample data and 29 deliberate constraint-violation checks support the implementation. Normalisation notes explain 1NF through 3NF using the actual functional dependencies and show why player, game, match and score details are stored separately.

Portfolio 2: Query Processing and Optimisation

Five SQL examples demonstrate joins, game lookup, per-match score counts, player rankings within a match and game-wide totals for completed matches. For an index comparison, I used a separate 50,000-row performance table and compared the same selective lookup before and after a B-tree index. The recorded median changed from 1.91 ms to 0.252 ms and the indexed plan used the new index while returning the same rows. These measurements describe that test workload and environment only.

Portfolio 3: Transactions and Concurrency

Using pgAdmin Query Tool and two database sessions, I tested `COMMIT`, `ROLLBACK` and `READ COMMITTED`. A deliberate check-constraint error aborted a transaction; rolling it back preserved the last committed counter value. The two-session

test showed a lost increment when both sessions wrote a value based on an earlier read: the final value was 2 rather than 3. Locking the row with `SELECT FOR UPDATE` made the second session wait, then apply its increment; the final value was 3. The counter is an exercise helper, not a business field in the Portfolio 1 design.

Portfolio 4: NoSQL and Advanced Data Models

The MongoDB database `portfolio_nosql` contains a `leaderboard_events` collection with four simulated `score_submitted` and `score_corrected` event documents. BSON dates, a JSON Schema validator, a compound event/time index, nested lookups, a date-range query and an aggregation demonstrate the document model. The corrected examples illustrate score 4001 changing from 120 to 135 and score 4002 from 85 to 90. They use the same case-study identifiers but are simulated events; the MongoDB script does not update or synchronize PostgreSQL.

Portfolio 5: Distributed and Cloud Database Design

I proposed a three-member MongoDB replica set across three availability zones in one region. The discussion covers primary elections, asynchronous replication, majority write and read concerns, secondary lag, availability limits, sharding trade-offs, security, backup and recovery. PostgreSQL requires its own high-availability plan; MongoDB replication does not replicate PostgreSQL. A transactional outbox and retrying publisher are proposed for future cross-database event delivery, but are not implemented. The architecture diagram is proposed-design evidence only; no cloud service, sharded cluster, or failover test was deployed.

Repository and Evidence

https://github.com/pfrederick3679/Frederick_Precious_Junior_301903679. The repository is organised into five portfolio folders and contains SQL/JavaScript, design notes, the ER diagram, query plans, test outputs and screenshots. The Portfolio 5 architecture image is explicitly marked as a proposal.

Individual Reflection

Separating player, game, match and score data reduced repeated descriptions and gave each relationship a clear key. The query exercise taught me to compare execution plans as well as returned rows and to limit performance claims to the workload measured. The

concurrency exercise made the lost-update problem visible and showed how row locking protects a read-modify-write sequence. MongoDB let me model score changes as event documents with related snapshots, while also making clear that a separate document database does not automatically stay in sync with PostgreSQL.

AI-use Declaration

I used the portfolio generator to assist with preparing code and documentation and ChatGPT/Codex to review the assessment requirements and refine documentation, query examples, event examples and the proposed architecture. I personally ran and tested the SQL and MongoDB code, reviewed the results and checked the AI-assisted explanations for consistency. Portfolio 5 remains a design proposal and was not deployed to a cloud service.

References

PostgreSQL: Constraints, Explain, Transaction, Isolation, Explicit locking.
MongoDB: Data Modelling, Replication, Sharding.
Mermaid ER diagrams.